**PL/SQL** 

**Static:** SQL whose full text is known at **`compile time`**

**Dynamic:** SQL whose full text is not known until **`run time => Flexible, Versatile`**

- PL/SQL is a **`procedural`** language extension for SQL that combines the data manipulation capabilities of SQL with the features of procedural programming.
- Statements can be organized into blocks that reduce traffic between the application and the database.
- PL/SQL also lets you invoke external programs written in other languages
- PL/SQL provides:
  - **`Blocks`**
  - Variables and Constants - Variabilele își pot schimba valoarea pe parcursul execuției; constantele nu.
  - Error Handling - Utilizarea secțiunii **`EXCEPTION`** pentru tratarea eficientă a erorilor.
  - Subprograms - **Proceduri** (Blocuri care nu returnează un rezultat ) și **Funcții** (Returnează un rezultat).
  - Packages - Obiecte care **grupează** logic subprograme, constante și tipuri de date.
  - Triggers - Unități PL/SQL care rulează **automat** la apariția unor evenimente în baza de date.
  - Input and Output
  - Data Abstraction - PL/SQL permite definirea propriilor structuri de date: **Cursors, Records, Collections**
  - Control Statements - Conditional ( **`IF`** și **`CASE`**), Loop (**`LOOP`, `FOR LOOP`** și **`WHILE LOOP`**), Sequential control **(`GOTO`)**
  - Conditional Compilation
  - Processing a Query Result Set One Row at a Time

- **Data Manipulation Language (DML):** Supports SQL operations: SELECT, INSERT, UPDATE, DELETE.
- **Transaction Control Language (TCL):** Manages transactions with commands: COMMIT, ROLLBACK, SAVEPOINT.

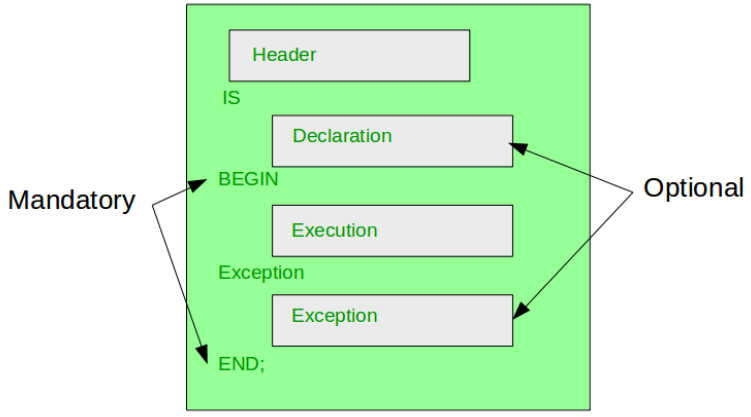
## PL/SQL Block

- A PL/SQL block is defined by the keywords:
  - DECLARE - declarative part
  - BEGIN - an executable part
  - EXCEPTION - an exception-handling part
  - END.
- The block is **not** stored in the database
- Declarations are local to the block; They cease to exist when the block completes execution
- Blocks can be nested
- Only the executable part is required (BEGIN ... END)
- A block can have a label. It is called an **anonymous block** (even if it has a label)
- There should be at least one instruction in the execution part of the block (BEGIN … END)
- Each instruction in a PL/SQL block ends with a semicolon (;). There is no semicolon after **DECLARE**, **BEGIN** and **EXCEPTION** .There is a semicolon after **END**
- To run the block in SQL*Plus use: Slash (/) și Run (r)

```
<< label >> (optional)
DECLARE    -- Declarative part (optional)
  -- Declarations of local types, variables, & subprograms

BEGIN      -- Executable part (required)
  -- Statements (which can use items declared in declarative part)

[EXCEPTION -- Exception-handling part (optional)
  -- Exception handlers for exceptions (errors) raised in executable part]
END;
```



**PL/SQL Identifiers:** constants, variables, exceptions, procedures, cursors, and reserved words

## Variables and Constants, Literals

- Variables in PL/SQL are declared using the **DECLARE** keyword within an anonymous block or a named program unit such as a procedure, function, or package.
- Scope:
  - **Local variables** − Declared within a block or subprogram,
  - accessible only inside that block or subprogram.
  - **Global variables** - Declared in the outermost block or package
- Initializing:
  - during declaration
  - after declaration

```
DECLARE
   variable_name datatype := initial_value;
```

- Constants are values used in a PL/SQL block that remain unchanged throughout the program.
- Constants are declared with the help of the **CONSTANT** keyword, followed by data type and initialization of value. It is also declared in the **DECLARE** part of a PL/SQL block

```
DECLARE
   constant_name CONSTANT data_type := value;
BEGIN
   -- Code block where the constant can be used
END;
```

- Literals are explicit numeric, character, string, or Boolean values not represented by an identifier.
- PL/SQL supports the following kinds of literals:
  - Numeric
  - Character
  - String
  - BOOLEAN
  - Date and Time

## Exception Handling in PL/SQL

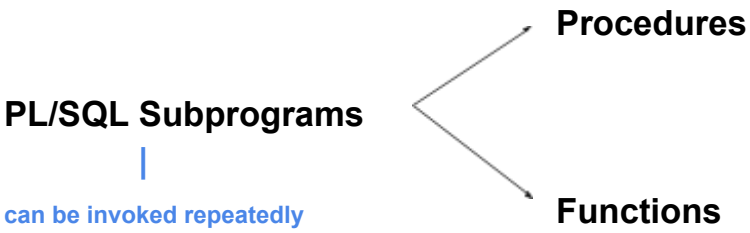
- They are PL/SQL runtime errors that can arise from
  - Design faults
  - Coding mistakes
  - Hardware failures
  - Many other sources
- There are two types of exceptions defined in PL/SQL:
  - User defined exceptions.
  - System defined exceptions:
    - Named system exceptions.
      1. **NO_DATA_FOUND:** It is raised WHEN a SELECT INTO statement returns no rows.
      2. **TOO_MANY_ROWS:** It is raised WHEN a SELECT INTO statement returns more than one row.
      3. **VALUE_ERROR:** This error is raised WHEN a statement is executed that resulted in an arithmetic, numeric, string, conversion, or constraint error.

    - Unnamed system exceptions

| Category | Definer | Has Error Code | Has Name | Raised Implicitly | Raised Explicitly |
|---|---|---|---|---|---|
| Internally defined | Runtime system | Always | Only if you assign one | Yes | Optionally |
| Predefined | Runtime system | Always | Always | Yes | Optionally |
| User-defined | User | Only if you assign one | Always | No | Always |

```
DECLARE
   declarations section;
BEGIN
   executable command(s);
EXCEPTION
    WHEN exception THEN
       statement;
    [WHEN others THEN]
    /* default exception handling code */
END;
```

```
DECLARE
   exception_name EXCEPTION;
BEGIN
   IF condition THEN
      RAISE exception_name;
   END IF;
EXCEPTION
   WHEN exception_name THEN
   statement;
END;
```




- Procedures are reusable blocks of code that contain a specific set of actions or logic.
- Procedures do not return a value directly; mainly used to perform an action.
- The procedure contains two parts:
  - Header: specifies the name and parameters
  - Body: contains the executable statements
- A procedure is created with the **CREATE** or **REPLACE** procedure statement.

```
CREATE [OR REPLACE] PROCEDURE procedure_name
[(parameter_name [IN | OUT | IN OUT] type [, ...])]
{IS | AS}
BEGIN
  < procedure_body >
END procedure_name;
```

- PL/SQL functions are reusable blocks of code that can be used to perform specific tasks. They are similar to procedures but must **always return** a value.
- The function contains two parts:
  - Header: includes the function name and an optional parameter list
  - Body: contains the executable statements
- To create a procedure in PL/SQL, use the **CREATE FUNCTION** statement.
- **PL/SQL functions** can be declared and defined in the **WITH** clauses of **SQL SELECT** statements

```
CREATE [OR REPLACE] FUNCTION function_name
[(parameter_name [IN | OUT | IN OUT] type [, ...])]
RETURN return_datatype
{IS | AS}
BEGIN
   < function_body >
END [function_name];
```

## PL/SQL Packages

- PL/SQL packages are a way to organize and encapsulate related procedures, functions, variables, triggers, and other PL/SQL items into a single item
- A package is compiled and then stored in the database, which then can be shared with many applications.
- The package contains two parts:
    - Specification: It includes declarations of procedures, **functions**, **variables**, **cursors**, and other constructs that are meant to be accessible from outside the package
    - Body: It includes the coding of the **procedures** or **functions** which are declared in the package specification. The body can also contain **private variables** and **procedures** that are not exposed to outside the code.
- They allow shared access across applications
- They also improve performance by loading once into memory

## PL/SQL Triggers

- Triggers are stored programs, which are **automatically** executed or fired when some events occur.
- Triggers are, in fact, written to be executed in response to any of the following events
    - A **database manipulation (DML)** statement (DELETE, INSERT, or UPDATE)
    - A **database definition (DDL)** statement (CREATE, ALTER, or DROP).
    - A **database operation** (SERVERERROR, LOGON, LOGOFF, STARTUP, or SHUTDOWN).

```
CREATE [OR REPLACE ] TRIGGER trigger_name
{BEFORE | AFTER | INSTEAD OF }
{INSERT [OR] | UPDATE [OR] | DELETE}
[OF col_name]
ON table_name
[REFERENCING OLD AS o NEW AS n]
[FOR EACH ROW]
WHEN (condition)
DECLARE
   Declaration-statements
BEGIN
   Executable-statements
EXCEPTION
```

*PL/SQL supports object-oriented programming through object types and Abstract Data Types*

## Cursors, Records, Collections

- When an SQL statement is processed, Oracle creates a memory area known as **context area**. A cursor is a **pointer** to this context area.
- The cursor is used to retrieve data **one row at a time** from the results set, unlike other SQL commands that operate on all rows at once.
- There are two types of cursors:
  - Implicit: **automatically generated** by Oracle when a query is executed. These are created by default to process the statements when **DML statements** like INSERT, UPDATE, DELETE etc. are executed.
  - Explicit: **defined by the programmers** to gain more control over the context area. These cursors should be defined in the declaration section of the PL/SQL block. It is created on a SELECT statement which returns **more than one row.**
- Cursor Actions:
  - Declare: A cursor is declared by defining the SQL statement that returns a result set.
  - Open: A Cursor is opened and populated by executing the SQL statement defined by the cursor.
  - Fetch: When the cursor is opened, rows can be fetched one by one or in a block

- In PL/SQL, **records** are structured data types that allow grouping multiple, related data fields under a single name.
- A record is a data structure that can hold data items of different kinds. Records consist of **different fields**, similar to a row of a database table.
- Types of records:
  - Table-Based: Linked to a table's row structure, allowing you to work directly with database table columns.
  - Cursor-Based: Derived from a cursor's query result set, providing a way to process query results one row at a time.
  - Programmer-Defined: User-defined records with custom fields and data types.
  - Nested-Records: Records within records, enabling hierarchical data management

- A collection is an ordered group of elements having the same data type.
- Collections have one dimension. A multidimensional collection can be modeled by using a collection whose elements are collections.
- There are three types:
  - Index-by tables or Associative array: a set of **key-value** pairs

```
TYPE type_name IS TABLE OF element_type [NOT NULL] INDEX BY subscript_type;

table_name type_name;
```

  - Nested table: like a one-dimensional array with an arbitrary number of elements

```
TYPE type_name IS TABLE OF element_type [NOT NULL];

table_name type_name;
```

  - Variable-size array or Varray: Collections that store elements in an ordered, indexed format, with a predefined maximum size.

*The ALTER TYPE statement allows you to modify a collection type defined at the schema level.*

| Non-PL/SQL Composite Type | Equivalent PL/SQL Composite Type |
|---|---|
| Hash table | Associative array |
| Unordered table | Associative array |
| Set | Nested table |
| Bag | Nested table |
| Array | VARRAY |

In **PL/SQL**, `%TYPE` and `%ROWTYPE` are **attribute types** that help define variables based on existing database columns or entire rows of a table.

- `%ROWTYPE` is used to declare a record that represents either a full or partial row of a database table or view.
- `%TYPE` allows you to declare a data item of the same data type as a previously declared variable or column, ensuring type compatibility.

# PL/SQL Performance

- PL/SQL performance focuses on optimizing the execution of SQL statements within PL/SQL blocks.
- Key strategies include using
    - bind variables:
    - leveraging the optimizer
    - employing dynamic SQL

**Bind Variables:**

- Placeholders for actual values in SQL statements.
- Allow reuse of SQL statements without reparsing or reanalyzing.
- Automatically created in static SQL; must be specified explicitly in dynamic SQL.

**Optimizer:**

- PL/SQL compiler includes an optimizer that rearranges code for improved performance.
- Enhances execution plans for SQL statements.

**Dynamic SQL**

- SQL whose full text is determined at runtime.
- Increases flexibility and versatility of applications.
- Requires careful handling of bind variables for optimal performance.
- Can lead to performance loss and security risks (e.g., SQL injection)
- Hard to debug

**Solutions for Dynamic SQL Security concerns**

- Limit database permissions for executing dynamic SQL
- Use stored procedures instead of dynamic SQL when possible
- Implement input validation to sanitize user inputs

## MS SQL Server

- It is a relational **database management system (RDBMS)** developed by Microsoft.
- It is **cloud-ready**
- Microsoft SQL Server or MS SQL Server for short is the query language provided for data definition and manipulation.
- MS SQL Server also has T-SQL or Transact-SQL and the main focus of T-SQL is to handle the transactions.
- Features:
    - Increased Performance **(memory-optimized tables, optimization strategies based on application runtime)**
    - High Availability **(HADR - High-Availability Disaster Recovery)**
    - Security **(Database snapshot - static view of database, Dynamic Data Masking - limits data exposures, encryption)**
    - Replication **(Three types of replication: Snapshot, Transactional, Merge)**
- Main Components:
    - Protocol Layer
    - Relational Layer
    - Storage Engine

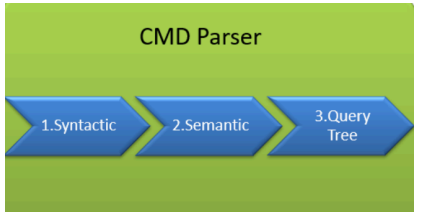
### Protocol Layer

- **Microsoft SQL Server Network Interface (SNI)**
- It uses on of the following protocols:
    - Shared memory - used to connect from a client running on the **same** computer as Microsoft SQL Server. **=> Only used locally**
    - TCP/IP
        - The most used access protocol or MS SQL Server. It is used for **local area networks (LANs)**
        - Enables connection to MS SQL Server by specifying an IP address and a port number
    - Named Pipes - used for **local area networks (LANs)**

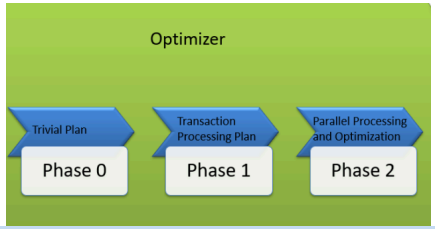
***All these protocols use Tabular Data Stream (TDS) which is a proprietary protocol developed by Microsoft for communication between a SQL Server instance and a client application***

**Relational Layer**

- Its primary function is **query optimization** and **execution**
- Called the query processor
  - It is responsible for the execution of user queries by **requesting data** from the storage engine and **processing the results.**
- Main responsibilities:
  - Parsing the SQL statements
  - Optimizing the execution plans
  - Executing the series of logical operations defined in the execution plan
  - Processing Data Definition Language (DDL) and other statements
  - Formatting results
- Has the following components:
  - CMD Parser (The Command Parser):Handles SQL and T-SQL language
    - Checks SQL syntax and semantics.
    - If valid, it creates a **query tree** (logical execution plan).
    - Looks for an existing execution plan in the **Plan Cache** to avoid re-compilation.




  - Optimizer : Determines the most efficient **execution plan** (Note that not all queries are optimized). It has three phases:
    - **Trivial Plan Search:** If there's only one execution method, it is used directly.
    - **Transaction Processing Plan:** Evaluates single-table and multi-table query plans.
    - **Parallel Processing**: Determines if parallel execution can improve performance.




  - Query Executor:
    - Executes the query.
    - Calls the Access Method



**Storage Engine**

- **It is responsible for managing all I/O to the data**
- Implementing utility functions such as the BACKUP, RESTORE
- Three components:
    - Access Methods Module:
        - **Interface between Query Executor and physical data storage through which data is retrieved and modified**
        - **Does not perform the operation itself. The operation is passed as a request to the Buffer Manager (`SELECT STATEMENT`) or to the Transaction Manager (`NOT A SELECT STATEMENT`)**

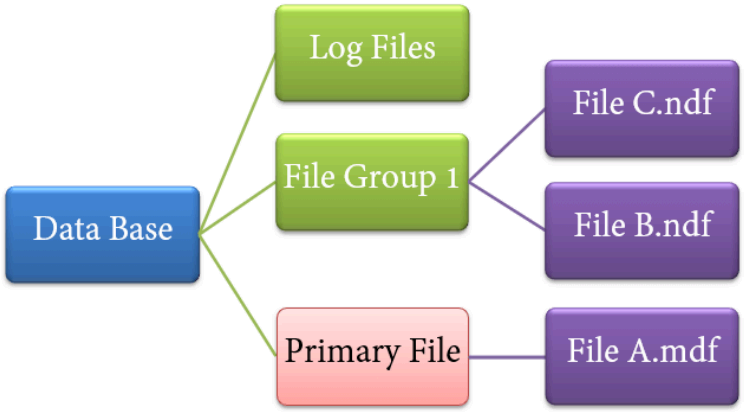
    - Buffer Manager: It has two mechanisms:
        - **The Buffer Manager** manages **data caching** in memory (reducing disk I/O).Used to access and update database pages:
            - Manages the Buffer Pool **(the majority of SQL Server's memory usage)**
            - Manages the core functions for:
                - Plan Cache: Stores execution plans
                - Data Cache:
                    - Stores table data
                    - Largest part of the Buffer Pool => largest memory consumer within SQL Server
                    - Implements **Soft Parsing** (retrieves data from cache) and **Hard Parsing** (retrieves data from disk).
                - Dirty Page: A page read from disk into memory is considered **clean**. If it has been modified in memory it is marked as a **dirty page**
        - **The Buffer Cache**: used to store the query plan for a SQL query and reduce database file I/O:

    - Transaction Manager: Two main components
        - Lock Manager : Ensures **data consistency** during concurrent access.
        - Log Manager : Tracks all changes for **rollback and recovery.**

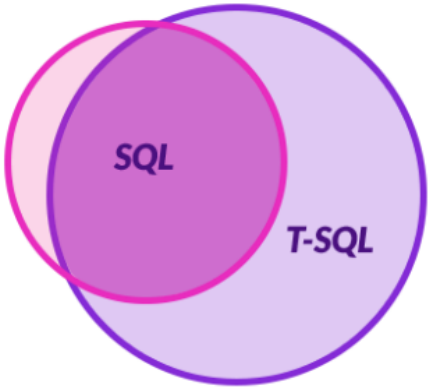
**Storage System**

- **Primary file:**
  - Every database contains one Primary file
  - This store all important data related to tables, views, Triggers, etc
- **Secondary file**
  - A Database may or may not contains multiple Secondary files
  - This is optional and contains user-specific data
- **Log file**
  - Also known as **Write Ahead Logs (WAL)**
  - Used for Transaction Management

Data is organized in **8 KB pages** and grouped into **extents (64 KB).**



## Transactions




- A transaction is a single unit of work
- If a transaction is successful, all the data modifications made during the transaction are **committed** and **become permanent**
- If a transaction encounters errors, all the data modifications made during the transaction are **erased** and **must be cancelled** or **rolled back**
- There are two types:
  - **Autocommit**: Are the default transaction management mode of the SQL Server Database Engine
  - **Explicit**: Are **explicitly started** with the BEGIN TRANSACTION statement and **explicitly ended** with a COMMIT `TRANSACTION` or `ROLLBACK TRANSACTION` statement
  - **Implicit:** A new transaction is implicitly started when the prior transaction completes

## Transact-SQL:

- It is a Microsoft SQL Server extension of SQL
- It contains:
  - Basic SQL clauses
  - Procedural programming abilities
  - Local variables

**SQL commands are categorized into different types based on their functionality:**

| Data Definition Language (DDL) | Data Control Language DCL | Data Manipulation Language (DML) |
|---|---|---|
| • CREATE<br>• ALTER<br>• DROP | • GRANT<br>• DENY<br>• REVOKE | • INSERT<br>• BULK INSERT<br>• UPDATE<br>• DELETE<br>• SELECT (considered DQL – Data Query Language) |

# The 4 (or 8) V's of Big Data:

1. **Volume**
    - Main characteristic
    - Volume of data impacts its analysis
2. **Variety**
    - types of data available for analysis
        i. Multiple types
        ii. Multiple sources
        iii. Structured, unstructured and hybrid
3. **Veracity** (cat de adevarata e)
    - Trustworthiness of the data
    - Data must be representative, relevant, consistent, accurate and current
4. **Velocity**
    - How fast the data is processed
5. **Value**
    - Increase revenue and decrease costs
6. **Variability**
    - Data is constantly changing, context where the data appears must be understood
7. **Visualization**
    - presentation of data in a pictorial or graphical form
8. **Vicissitude**
    - challenges of scaling Big Data

**Why use Big Data now**:

- Increased storage capacities
- Increase of processing power
- Availability of data
    - More data is available
    - Different types of data are available

## Types of Big Data

- **Structured Big Data**
  - Data that have a predefined data model
  - Has a strict schema
  - data set's attributes are predefined and have a strict data type: number, datetime, string
  - Machine or human generated
  - Structured Big Data technologies:
    - Relational databases: Oracle, PostgreSQL, MySQL, Microsoft SQL server, Teradata
    - Data Warehouses and Data Marts
    - Enterprise Resource Planning software (ERP)
    - Customer Relationship Management software (CRM)
    - Mainframes
- **Unstructured Big Data**
  - Data that does not have a predefined data model
  - Does not have a strict schema
  - Data set's attributes are not predefined and they do not have a strict data type
  - Machine or human generated
  - Unstructured Big Data technologies:
    - NoSQL databases: MongoDB, CouchDB, Neo4J, Cassandra, etc.
    - Distributed storage frameworks: Hadoop
    - Data Lakes (a storage repository that stores vast amounts of raw data in its native format).
    - Enterprise Content Management Systems (CMS) that manage the complete life cycle of content
- **Hybrid (Multi-structured/Semi-structured) Big Data**
  - Combines Structured and Unstructured Big Data (Combines data sources, technologies, everything)

**When is Big Data a problem:**

- When the operations on the data are complex:
- When usual algorithms cannot handle big amounts of data:

**Distributed architectures:**

- A distributed architecture is a model in which resources located on network computers are put together in a resource pool to achieve a common goal.
- A cluster is a set of tightly connected computers that work together so that they can be seen as a single system
- Scalability is the capability of a system to handle a growing amount of work
- **Types of scaling:**
  - Vertical scaling (scale up/down):
    - You can scale by adding more resources to an existing machine, e.g., CPU, RAM, HDD, etc.
  - Horizontal scaling (scale out/in):
    - You can scale by adding more machines into the resource pool, e.g., add nodes to the cluster.
- **Resource sharing in a cluster:**
  - Shared everything architecture - Shared disk storage, memory, cpu usage
  - Shared nothing architecture - Independent and self sufficient nodes
- **Replication** involves sharing information to
  - Ensure consistency between redundant resources
  - Improve reliability, fault-tolerance, or accessibility
- **Replication** is archived when the same data is stored on multiple storage devices
- **Replication from the architecture perspective:**
  - Master-Slave (Primary-Secondary) : Master face tot, Slave citeste
  - Multi-Master (Multi-Primary) : Toate fac tot
- **Cloud Computing**
  - – Infrastructure as a Service (IaaS): Virtual computing resources over the internet
  - – Database as a Service (DBaaS): Baza de date dar pe net lol
  - – Platform as a Service (PaaS): All the hardware and software resources to develop applications
  - – Software as a Service (SaaS): Program dar pe net